Università degli Studi di Catania

Dipartimento di Matematica e Informatica

Corso di Artificial and Swarm Intelligence
Relazione Progetto

Warehouse Swarm Intelligence System

Simulazione Multi-Agente per il Recupero di Oggetti
in Ambienti di Magazzino

Autore:
Giuseppe Bellamacina
Matricola: 1000030349

Professore del corso:
Mario Francesco Pavone

Anno Accademico 2025/2026

Indice

1 Struttura Comune: BaseAgent **2**
1.1 Percezione e Mappa Locale 2
1.2 Integrazione della Mappa Condivisa 2
1.3 Protocollo ClearWay 2
1.4 Rilevamento dello Stallo 3

2 Scout Agent **4**
2.1 Ciclo Decisionale 4
2.2 Anti-Clustering 5

3 Coordinator Agent **6**
3.1 Raccolta delle Informazioni 6
3.2 Logica di Assegnazione 6
3.3 Riposizionamento verso il Centroide 7
3.4 Sincronizzazione tra Coordinator Multipli 7

4 Retriever Agent **8**
4.1 Gerarchia delle Priorità 8
4.2 Sequenza di Consegna al Magazzino 9
4.3 Selezione del Magazzino Ottimale 9
4.4 Peer Negotiation tra Retriever 10

Capitolo 1

Struttura Comune: BaseAgent

Tutti e tre i tipi di agente ereditano da `BaseAgent`, che implementa le funzionalità condivise: percezione dell'ambiente, ricezione e invio messaggi, movimento tramite A*, gestione dell'energia e rilevamento dello stato di stallo.

1.1 Percezione e Mappa Locale

Ogni agente mantiene una *mappa locale* rappresentata come matrice numpy, inizialmente tutta `UNKNOWN`. Ad ogni passo, la fase `STAGE_SENSE` aggiorna le celle entro il raggio di visione con il tipo reale della griglia, applicando la logica di *ray casting*: le celle dietro un ostacolo non vengono aggiornate anche se geometricamente entro il raggio, simulando la visibilità lineare degli agenti fisici. Gli oggetti nella zona di visione vengono aggiunti alla lista `known_objects`; le celle magazzino vengono aggiunte a `known_warehouses`.

1.2 Integrazione della Mappa Condivisa

Alla ricezione di un `MapDataMessage`, l'agente aggiorna le celle della propria mappa ancora `UNKNOWN` con i dati del mittente. Il merge estrae anche le celle magazzino, popolando `known_warehouses`: in questo modo i Retriever scoprono le stazioni di ricarica ben prima di visitarle, purché uno Scout o un Coordinator vicino le abbia già esplorate.

1.3 Protocollo ClearWay

Quando un agente calcola un percorso A* verso il proprio obiettivo ma la cella successiva è occupata da un altro agente fermo, anziché attendere o

ricalcolare un percorso alternativo, invia un `ClearWayMessage` direttamente all'occupante. L'agente ricevente, se libero da vincoli immediati, si sposta sulla cella adiacente più vicina libera. Questa negoziazione è ricorsiva (`depth` nel messaggio) fino a una profondità massima: se la catena di spostamenti non si risolve, l'agente iniziante rilascia il messaggio e lascia che A* trovi una via alternativa al passo successivo. Il meccanismo riduce significativamente i blocchi in corridoi stretti senza richiedere alcuna struttura di prenotazione globale.

1.4 Rilevamento dello Stallo

L'agente registra lo storico delle posizioni recenti. Se per un certo numero di passi consecutivi (`stuck_threshold`) la posizione non cambia, il campo `stuck_counter` viene incrementato. Superata la soglia, l'agente esegue una *stuck recovery*: invalida il percorso corrente, aggiunge le posizioni degli agenti vicini come evitate e ricalcola un percorso alternativo verso una destinazione leggermente ruotata. Se lo stallo persiste, l'agente si sposta temporaneamente verso una cella casuale libera per sbloccarsi.

Quando il rilevatore di loop o di stallo abbandona un target, la posizione viene inserita nella blacklist `unreachable_targets` con il passo corrente: in questo modo il ciclo decisionale dell'agente non riassegna immediatamente lo stesso obiettivo irraggiungibile nel passo successivo.

Capitolo 2

Scout Agent

Lo Scout è ottimizzato per la copertura massima della griglia nel minor numero di passi. Non recupera oggetti e non riceve compiti: la sua unica responsabilità è esplorare e condividere informazioni.

2.1 Ciclo Decisionale

La priorità dello Scout è deterministica e segue questa sequenza:

1. **Ricarica energetica**: Se l'energia scende sotto la soglia critica (calcolata come la distanza stimata al magazzino più vicino moltiplicata per un fattore di sicurezza), lo Scout interrompe l'esplorazione e si dirige al magazzino. Durante la ricarica, continua a comunicare la mappa locale e gli oggetti scoperti.

2. **Consegna scoperte al Coordinator**: Se lo Scout ha scoperto nuovi oggetti e un Coordinator si trova nel raggio di comunicazione, li trasmette immediatamente tramite `ObjectSpottedMessage`. Se nessun Coordinator è a portata, lo Scout si dirige verso l'ultima posizione nota del Coordinator (`last_seen_coordinator_pos`) per consegnare le informazioni al più presto. Se è trascorso troppo tempo senza trovarlo, viene effettuata una scansione estesa fino a $4\times$ il raggio di comunicazione per localizzarlo.

3. **Selezione frontiera**: Se non è in ricarica, `FrontierExplorer` calcola le frontiere disponibili dalla mappa locale e seleziona quella con utilità massima. La funzione $U(f)$ favorisce cluster grandi e vicini, penalizzando cluster già puntati da altri Scout (comunicazione implicita via `target_position` osservata nei messaggi di stato).

4. **Movimento**: A* calcola il percorso ottimale verso la frontiera selezionata, con le posizioni degli altri agenti come penalizzazione soft.

5. **Comunicazione**: Ad ogni passo, lo Scout condivide con tutti gli agenti entro il raggio `comm_radius` la propria mappa locale aggiornata tramite `MapDataMessage` e la lista degli oggetti scoperti tramite `ObjectSpottedMessage` indirizzati ai Coordinator vicini.

2.2 Anti-Clustering

L'anti-clustering è implementato in due livelli. Il primo è un filtro hard: le frontiere che si trovano a meno di 8 celle da un altro Scout (visibile nel raggio di comunicazione) vengono escluse completamente dall'insieme dei candidati. Il secondo è una penalizzazione soft nella funzione di utilità:

$$U(f) = \frac{\text{cluster_size}}{d(f) + 1} - 0{,}5 \cdot n_{\text{agenti}}(f)$$

dove $n_{\text{agenti}}(f)$ è il numero di agenti entro 10 celle dalla frontiera f. Se il filtro hard elimina tutte le frontiere (caso raro, ad esempio con molti Scout su una griglia piccola), si ricade sull'insieme completo applicando solo la penalizzazione soft. Il risultato pratico è una divisione organica del territorio senza alcuna coordinazione esplicita.

Capitolo 3

Coordinator Agent

Il Coordinator è l'unico agente con visione globale *aggregata*: non può vedere tutta la griglia, ma riceve informazioni da tutti gli Scout vicini e costruisce un modello del mondo più completo rispetto ai Retriever. Il suo ruolo è pianificare chi deve recuperare cosa, evitando ridondanze e massimizzando l'efficienza del sistema.

3.1 Raccolta delle Informazioni

Ogni volta che riceve un `ObjectSpottedMessage` da uno Scout, il Coordinator aggiorna il proprio `known_objects` con la posizione e il valore dell'oggetto. Ogni `TaskStatusMessage` ricevuto da un Retriever aggiorna uno dei dizionari interni: `retriever_carrying` (numero di oggetti trasportati), `retriever_energy` (energia residua) e `objects_being_collected` (set degli oggetti già in recupero). Il set `objects_being_collected` è la protezione principale contro la doppia assegnazione: qualsiasi oggetto già assegnato o dichiarato in recupero da un Retriever viene escluso dalla lista di candidati per le nuove assegnazioni.

3.2 Logica di Assegnazione

La funzione `_plan_task_assignments` calcola per ogni coppia (Retriever, oggetto) una priorità greedy:

$$\text{priority}(o, r) = \frac{\text{value}(o)}{d(r, o) + 1} + b_{\text{spotted}}$$

dove $b_{\text{spotted}} = 0{,}5$ se il Retriever ha già visto l'oggetto direttamente (campo `_spotted_by`), altrimenti 0. I candidati vengono ordinati per

priorità decrescente e assegnati in modo greedy rispettando i slot liberi di ciascun Retriever. Al momento dell'invio, il Coordinator trasmette sia il `TaskAssignmentMessage` che un `MapDataMessage` con le celle magazzino note, accelerando la scoperta delle stazioni di deposito da parte dei Retriever che non le hanno ancora esplorate.

3.3 Riposizionamento verso il Centroide

Quando il Coordinator non ha compiti da assegnare, il suo obiettivo di movimento è rimanere raggiungibile dalla propria flotta. All'avvio, finché non riceve nessuna posizione recente da un Retriever, il Coordinator percorre un circuito di cinque waypoint strategici (centro e quattro quadranti della griglia, agganciati alla cella libera più vicina) per coprire tutta la griglia e ristabilire il contatto nel minor numero di passi.

Non appena arrivano posizioni fresche (più recenti di 25 passi), calcola il centroide medio di tutti i Retriever e, se la propria distanza da quel punto supera una soglia (pari a $\frac{1}{4}$ del raggio di comunicazione), si sposta verso la cella libera più vicina al centroide.

Se la distanza è già entro la soglia, il Coordinator rimane fermo (`IDLE`): non esegue nè esplorazione frontier-based nè passeggiate casuali, poichè il suo compito non è esplorare ma coordinarsi. Questa scelta riduce il consumo energetico e evita che l'agente si allontani involontariamente dagli agenti che gestisce.

Se esistono oggetti noti ma nessun Retriever è nel raggio di comunicazione, il Coordinator entra in modalità **SEEK-RETRIEVER**: si dirige verso l'ultima posizione dichiarata di un Retriever noto, con l'obiettivo di ristabilire il canale di comunicazione e procedere con l'assegnazione appena possibile.

3.4 Sincronizzazione tra Coordinator Multipli

In configurazioni con più di un Coordinator, il rischio di doppia assegnazione aumenta. Il `CoordinatorSyncMessage` viene scambiato periodicamente tra Coordinator vicini e contiene la lista delle assegnazioni attive (`assigned_tasks`) e degli oggetti in recupero (`objects_being_collected`). Alla ricezione, ogni Coordinator fonde le informazioni ricevute con le proprie, aggiornando i set interni. Questa sincronizzazione non è istantanea — si propaga a onde tramite la comunicazione locale — ma è sufficiente per eliminare la stragrande maggioranza delle doppie assegnazioni in scenari realistici.

Capitolo 4

Retriever Agent

Il Retriever è l'agente più complesso: deve recuperare oggetti fisicamente sulla griglia, gestire la capacità di carico, navigare verso il magazzino, ricaricarsi e uscire, tutto mantenendo una comunicazione attiva con Coordinator e altri Retriever.

4.1 Gerarchia delle Priorità

Il ciclo decisionale del Retriever è una gerarchia di priorità decrescenti, valutate in sequenza ad ogni passo della fase `STAGE_DECIDE`:

1. **P1 — Consegna**: Se trasporta almeno un oggetto, si dirige al magazzino tramite la sequenza di consegna. Questa priorità non è mai interrotta da nuovi compiti assegnati: il Retriever completa sempre la consegna prima di iniziare il prossimo recupero.

2. **P2 — Ricarica d'emergenza**: Se l'energia scende sotto la soglia critica e non sta già consegnando, il Retriever interrompe qualsiasi operazione e si dirige al magazzino per ricaricarsi. La soglia è dinamica: se il magazzino più vicino noto dista d celle, la soglia di allarme è $d \times 1.5$ unità di energia, garantendo sempre un margine di sicurezza.

3. **P3 — Esecuzione compiti assegnati**: Se ha una coda di compiti assegnati dal Coordinator, preleva il prossimo oggetto dalla coda e si dirige verso di esso.

4. **P4 — Auto-assegnazione (modalità hive-mind)**: In assenza di compiti assegnati, il Retriever scansiona l'intero `known_objects` accumulato (non solo le celle visibili) e si auto-assegna l'oggetto più conveniente tramite `try_claim_object` atomico. Questo meccanismo elimi-

na i tempi morti tra un'assegnazione e l'altra, sfruttando la conoscenza condivisa tramite le mappe ricevute dagli altri agenti.

5. **P5 — Mini-esplorazione**: Se non ha né compiti né oggetti noti, il Retriever esegue una breve frontier-based exploration locale per scoprire oggetti nell'area circostante.

4.2 Sequenza di Consegna al Magazzino

La navigazione al magazzino avviene in quattro sotto-fasi gestite da una macchina a stati interna (`_wh_step`):

1. **Approach**: Il Retriever si dirige verso la cella `WAREHOUSE_ENTRANCE` più vicina del magazzino ottimale (selezionato da `get_best_warehouse_for`). Il percorso A* usa le forbidden types per forzare il passaggio dall'ingresso, escludendo le celle di uscita come nodi intermedi.

2. **Deposit**: Entrato nel magazzino attraverso l'ingresso, il Retriever deposita tutti gli oggetti trasportati, che vengono conteggiati nelle metriche e rimossi dalla griglia. Lo stato passa a `RECHARGE` se l'energia è sotto soglia, altrimenti direttamente a `EXIT`.

3. **Recharge**: Il Retriever rimane sulla cella magazzino mentre viene ricaricato a tasso costante (`recharge_rate` unità per passo) fino al recupero completo dell'energia massima. Durante la ricarica continua a ricevere messaggi.

4. **Exit**: Il Retriever si dirige verso la cella `WAREHOUSE_EXIT` e la attraversa. Uscito dal magazzino, `_wh_step` viene azzerato; prima di tornare all'esplorazione, il Retriever tenta immediatamente una nuova auto-assegnazione tramite `known_objects`, riducendo i passi a vuoto tra una consegna e la successiva.

4.3 Selezione del Magazzino Ottimale

La funzione `get_best_warehouse_for` seleziona la stazione di deposito/ricarica ottimale tra tutte quelle note all'agente. Il punteggio per ogni stazione combina tre fattori:

- **Distanza**: La distanza euclidea dalla posizione corrente del Retriever.

- **Capacità**: Le stazioni con più Retriever già in coda vengono penalizzate per distribuire il carico.
- **Fattibilità energetica**: Se la distanza stimata (moltiplicata per un fattore di percorso 1.4 per tenere conto degli ostacoli) supera l'energia residua dell'agente, la stazione riceve una penalizzazione aggiuntiva di 200 punti, rendendola di fatto non preferibile. Questo impedisce al Retriever di dirigersi verso un magazzino irraggiungibile e rimanere senza energia a metà percorso.

4.4 Peer Negotiation tra Retriever

I conflitti tra Retriever che puntano allo stesso oggetto si risolvono in due livelli:

1. **Claim atomico**: Ogni auto-assegnazione passa per `CommunicationManager.try_claim_c` operazione atomica condivisa da tutti gli agenti. Solo un Retriever ottiene il claim; i contendenti ricevono immediatamente un fallimento e selezionano un altro oggetto.
2. **Aggiornamento reattivo via TaskStatusMessage**: Dopo ogni claim riuscito, il Retriever diffonde un `TaskStatusMessage` a tutti i vicini. Alla ricezione, ogni peer rimuove dalla propria `task_queue` gli oggetti già presenti nella coda del mittente (`PEER-YIELD`), aggiornando preventivamente la propria pianificazione senza attendere un fallimento esplicito.

L'assenza di un arbitro centralizzato e la natura broadcast del `TaskStatusMessage` garantiscono convergenza rapida: nella pratica, le collisioni residue dopo il claim atomico sono rare e vengono risolte entro uno-due passi di comunicazione.